

1. Bubble Sort

Bubble Sort adalah algoritma pengurutan sederhana yang berulang kali membandingkan

Pseudocode:

```
procedure bubbleSort( A : list of sortable items )
    n = length(A)
    repeat
        swapped = false
        for i = 1 to n-1 inclusive do
            if A[i-1] > A[i] then
                swap(A[i-1], A[i])
                swapped = true
            end if
        end for
        n = n - 1
    until not swapped
end procedure
```

```
#kode python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Analisis Kompleksitas: Waktu (Time Complexity):

- Best Case: $O(n)$ (jika array sudah terurut).
- Average & Worst Case: $O(n^2)$ (karena menggunakan dua loop bersarang).
- Ruang (Space Complexity): $O(1)$ (hanya membutuhkan memori tambahan untuk variabel swapped dan proses swap).

2. Selection Sort Selection sort bekerja dengan membagi array menjadi dua bagian: bagian yang sudah terurut dan yang belum. Algoritma ini mencari elemen terkecil dari bagian yang belum terurut dan menukarnya dengan elemen pertama di bagian tersebut.

Pseudocode:

```
procedure selectionSort( A : list of sortable items )
    n = length(A)
    for i = 0 to n-1 do
        minIndex = i
        for j = i+1 to n do
```

```

        if A[j] < A[minIndex] then
            minIndex = j
        end if
    end for
    if minIndex != i then
        swap(A[i], A[minIndex])
    end if
end for
end procedure

```

```

#kode python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

```

Analisis Kompleksitas: Waktu (Time Complexity):

- Best, Average, & Worst Case: $O(n^2)$ (karena selalu mencari elemen minimum di sisa array tanpa mempedulikan urutan awal).
- Ruang (Space Complexity): $O(1)$ (pengurutan dilakukan in-place).

3. Insertion Sort Insertion sort membangun array yang terurut satu per satu dengan mengambil elemen dari data yang belum terurut dan menyisipkannya pada posisi yang tepat di bagian data yang sudah terurut.

Pseudocode:

```

procedure insertionSort( A : list of sortable items )
    n = length(A)
    for i = 1 to n-1 do
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key do
            A[j+1] = A[j]
            j = j - 1
        end while
        A[j+1] = key
    end for
end procedure

```

```

#Kode Python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1

```

```
    arr[j + 1] = key
return arr
```

Analisis Kompleksitas:Waktu (Time Complexity):

- Best Case: $O(n)$ (jika array sudah terurut, loop while tidak dieksekusi).
- Average & Worst Case: $O(n^2)$ (ketika elemen harus digeser secara maksimal).
- Ruang (Space Complexity): $O(1)$ (pengurutan dilakukan in-place).

4. Merge Sort

Merge Sort menggunakan paradigma Divide and Conquer. Algoritma ini memecah a

Pseudocode:

```
procedure mergeSort( A : list of sortable items )
    if length(A) <= 1 then
        return A
    end if

    mid = length(A) / 2
    left = A[0...mid-1]
    right = A[mid...end]

    return merge(mergeSort(left), mergeSort(right))
end procedure

procedure merge( left, right )
    result = empty list
    while left is not empty and right is not empty do
        if first(left) <= first(right) then
            append first(left) to result
            left = rest(left)
        else
            append first(right) to result
            right = rest(right)
        end if
    end while

    append remaining elements of left and right to result
    return result
end procedure
```

```
#kode python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] <= R[j]:
                arr[k] = L[i]
                i += 1
```

```

        else:
            arr[k] = R[j]
            j += 1
            k += 1

    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1

    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

    return arr

```

Analisis Kompleksitas: Waktu (Time Complexity):

- Best, Average, & Worst Case: $O(n \log n)$ (karena array selalu dibagi dua dan proses merge memakan waktu linier).
- Ruang (Space Complexity): $O(n)$ (membutuhkan memori tambahan untuk menyimpan sub-array L dan R).